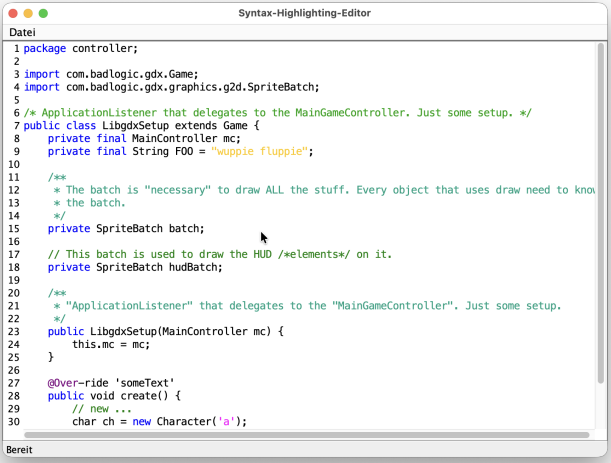
Einführung in ANTLR

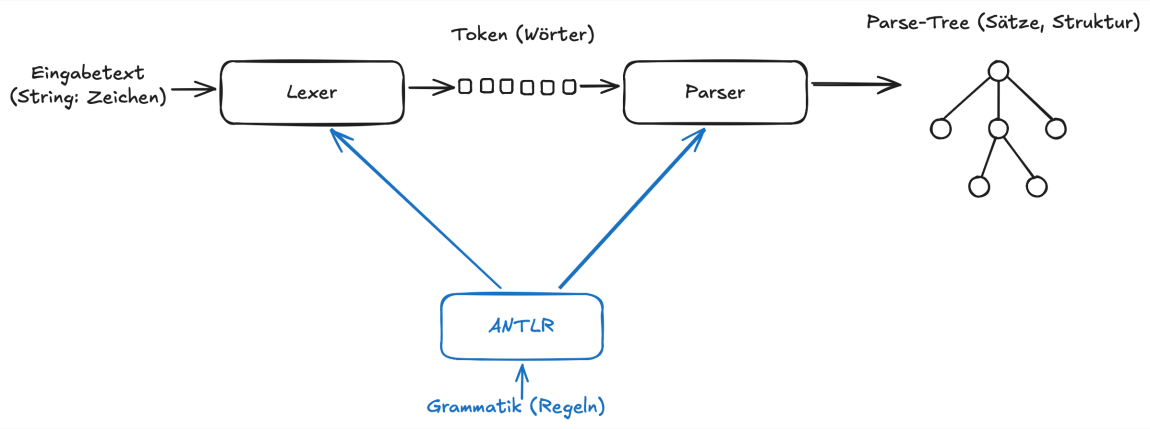
Carsten Gips (HSBI)

Motivation & Einordnung



Von regulären Ausdrücken zu “richtigen” Bäumen: ANTLR

ANTLR als Blackbox-Pipeline



Technische Einbindung (Gradle, Projektstruktur)

```
plugins {
    id 'java'
    id 'antlr'
}

repositories {
    mavenCentral()
}

dependencies {
    antlr 'org.antlr:antlr4:4.13.2'
    implementation 'org.antlr:antlr4-runtime:4.13.2'
}
```

Beispielgrammatik

```
grammar MiniCalc;

// Programm besteht aus beliebig vielen Statements
prog  : stmt* EOF ;

// Statement: entweder Zuweisung oder nackter Ausdruck
stmt  : ID '=' expr ';'
      | expr ';'
      ;

// Ausdruck: eine oder mehrere Zahlen, addiert
expr  : INT ('+' INT)* ;

// LEXER-Regeln (Token)
ID    : [a-z][a-zA-Z0-9_]* ;
INT   : [0-9]+ ;

WS    : [ \t\r\n]+ -> skip ;
```

Minimaler Java-Code: Zerlegen der Eingabe in Token

```
var input = CharStreams.fromString(text);
var lexer = new MiniCalcLexer(input);
var tokens = new CommonTokenStream(lexer);

tokens.fill(); // fill stream (fetch all tokens from lexer)

for (var t : tokens.getTokens()) {
    var tokenName = MiniCalcLexer.VOCABULARY.getSymbolicName(t.getType());
    System.out.printf(
        "%-10s line=%d col=%d text='%s'%n",
        tokenName, t.getLine(), t.getCharPositionInLine(), t.getText());
}
```

Minimaler Java-Code: Text -> Baum

```
var input = CharStreams.fromString(text);
var lexer = new MiniCalcLexer(input);
var tokens = new CommonTokenStream(lexer);

var parser = new MiniCalcParser(tokens);
var tree = parser.prog(); // Wurzelknoten des Baums (Startregel der Grammatik)

IO.println(tree.toStringTree(parser));
```

Der Parse-Tree: Struktur

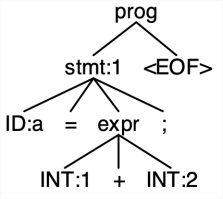
```
grammar MiniCalc;

prog  : stmt* EOF ;
stmt  : ID '=' expr ';'
      | expr ';'
      ;
expr  : INT ('+' INT)* ;

ID    : [a-z][a-zA-Z0-9_]* ;
INT   : [0-9]+ ;
WS    : [ \t\r\n]+ -> skip ;
```

Eingabe `a = 1 + 2;` liefert:

```
(prog (stmt a = (expr 1 + 2) ;) <EOF>)
```



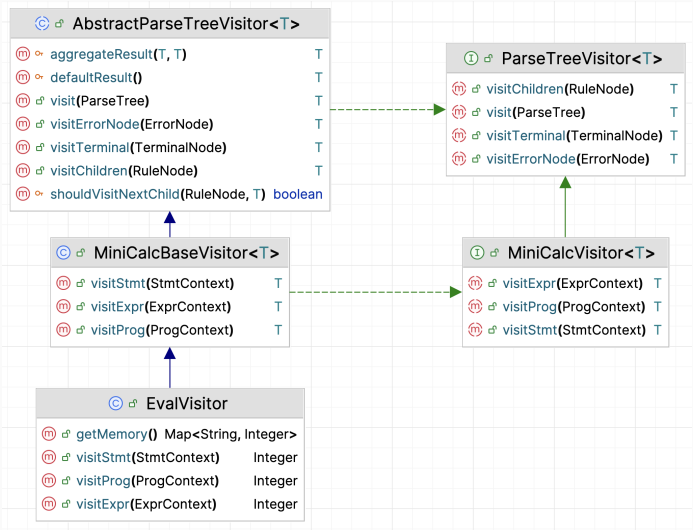
Der Parse-Tree: Klassenhierarchie

```
grammar MiniCalc;

prog  : stmt* EOF ;
stmt  : ID '=' expr ';'
      | expr ';'
      ;
expr  : INT ('+' INT)* ;

ID    : [a-z][a-zA-Z0-9_]* ;
INT   : [0-9]+ ;
WS    : [ \t\r\n]+ -> skip ;
```

Traversierung mit Visitor-Pattern



Wrap-Up

- ANTLR als Werkzeug: aus einer gegebenen Grammatik werden Lexer, Parser, Kontextklassen und Visitor-Schnittstellen generiert
- Einbettung in ein Java/Gradle-Projekt:
  - `antlr`-Plugin in `build.gradle`
  - `.g4`-Dateien unter `src/main/antlr/`
  - generierte Sourcen im Build-Ordner
- Üblicher Workflow: Eingabetext -> CharStream -> Lexer -> TokenStream -> Parser -> Parse-Tree
- Struktur des Parse-Baums:
  - jedes Knotenobjekt ist eine `XxxContext`-Klasse
  - Blätter sind Token
  - Whitespaces/Kommentare werden i.d.R. übersprungen/entfernt
- Visitor-Pattern in ANTLR:
  - eigene Klasse von generiertem `FooBaseVisitor<T>` ableiten
  - `visitXxx`-Methoden bei Bedarf überschreiben
  - Baum mit `visitor.visit(tree)` traversieren.

Last modified: *44971f0 2026-07-01 antlr: fix alert boxes*